

Top-k diversified vector search

Yuanchao Yang



4th Year Project Report
Artificial Intelligence
School of Informatics
University of Edinburgh

2026

Abstract

Dense vector retrieval systems usually optimise only query similarity, which often produces highly redundant top- k results. This is especially problematic for ambiguous or exploratory queries, where users benefit from coverage across multiple semantic facets rather than repeated variants of the same answer. This dissertation studies *top-k diversified vector search*, where the goal is to return a set of k results that is both relevant to the query and non-redundant.

I propose a query-adaptive two-stage pipeline. First, an approximate nearest neighbour (ANN) retriever generates a candidate pool of size N_c . Second, an incremental Maximal Marginal Relevance (MMR) reranker selects the final list. Before retrieval, the system combines a learned intent regressor with factual fallback rules and temporal cues to estimate query ambiguity and recency sensitivity. These signals choose between IVF and HNSW, adjust search width and candidate-pool size, and set the reranking parameters λ and β for relevance, diversity, and freshness.

To keep reranking practical, the incremental MMR implementation uses $O(N_c)$ memory rather than storing the full $O(N_c^2)$ candidate–candidate similarity matrix. The reranker is further extended with a relevance floor, a quality-aware fallback, a lightweight freshness prior, and a cluster-aware penalty for ambiguous IVF queries. As a comparison baseline, the evaluation also includes a score-based adaptive- λ method derived from the observed candidate score distribution.

Experiments on passage retrieval show the intended query-sensitive behaviour: ambiguous queries are often broadened, whereas specific and factual queries remain closer to a relevance-first regime. Fixed-MMR remains the strongest simple baseline on the current evaluation set, but the full adaptive pipeline provides the strongest overall adaptive compromise among the proposed variants while operating within a practical ANN serving architecture.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(Yuanchao Yang)

Acknowledgements

I would like to thank my supervisor for guidance, feedback, and support throughout this project. I am also grateful to friends and classmates who discussed ideas and helped test intermediate versions of the system.

Table of Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Project Overview and Key Idea	2
1.3	Contributions	3
2	Background	4
2.1	Vector Similarity Search	4
2.2	Approximate Nearest Neighbour Indexes	5
2.2.1	Inverted File (IVF) Indexes	5
2.2.2	Graph-Based ANN: HNSW	5
2.3	Diversification and MMR	5
2.4	Evaluation Metrics for Relevance and Diversity	6
3	System Design	8
3.1	System Overview	8
3.2	Three-Signal Query Analysis	9
3.2.1	Learned intent model	10
3.2.2	Factual fallback rules	11
3.2.3	Temporal detection	11
3.2.4	Fusion into control parameters	12
3.3	Dynamic Index Selection	13
3.4	Adaptive ANN Search Parameters	13
4	Candidate Retrieval Backends	15
4.1	Brute-force retrieval	15
4.2	IVF index	15
4.2.1	Training	16
4.2.2	Search	16
4.2.3	Cluster-balanced sampling	16
4.3	HNSW index	17
4.3.1	Construction	17
4.3.2	Search	17
4.4	Summary	18
5	Diversified Re-ranking	19
5.1	Standard MMR	19

5.2	Incremental MMR for efficiency	20
5.3	Relevance floor	21
5.4	Cluster-aware penalty	21
5.5	Temporal MMR	22
5.6	Quality-aware graceful degradation	22
5.7	Score-based adaptive- λ baseline	22
5.8	Complexity summary	24
6	Implementation	25
6.1	Codebase structure	25
6.2	Intent model implementation	25
6.3	Serving pipeline in the demo	26
6.4	Key data structures	26
7	Experiments and Evaluation	27
7.1	Evaluation philosophy and experimental setup	27
7.2	Metrics used in this chapter	28
7.3	Overall end-to-end quantitative results	29
7.4	Ablation study	30
7.5	Index-level retrieval experiments	32
7.6	Intent model evaluation	33
7.7	Qualitative case studies	34
7.7.1	Ambiguous queries	35
7.7.2	Specific queries	35
7.8	Discussion	36
8	Conclusions and Future Work	37
A	Qualitative case study screenshots	39
	Bibliography	43

Chapter 1

Introduction

Dense vector retrieval is now a standard component in search, recommendation, and retrieval-augmented systems (Karpukhin et al., 2020; Covington et al., 2016; Lewis et al., 2020). In a bi-encoder setting, queries and documents are embedded independently into a shared semantic space, allowing document embeddings to be pre-computed offline and searched efficiently at serving time (Reimers and Gurevych, 2019; Karpukhin et al., 2020). Large collections are typically searched with approximate nearest neighbour (ANN) indexes such as inverted file (IVF) structures and Hierarchical Navigable Small World (HNSW) graphs, which reduce latency while preserving high recall (Johnson et al., 2017; Jégou et al., 2011; Malkov and Yashunin, 2020). This project uses a Sentence-Transformers encoder based on MiniLM, which provides semantically useful sentence embeddings at low inference cost (Reimers and Gurevych, 2019; Wang et al., 2020; Sentence-Transformers).

Relevance-only top- k vector retrieval, however, often returns highly redundant results. Nearby vectors frequently correspond to paraphrases, near-duplicates, or items from the same local semantic neighbourhood. This is especially problematic for ambiguous or exploratory queries such as *spring* or *mercury*, where users may prefer coverage across several plausible interpretations rather than repeated evidence for one dominant sense. Search-result diversification addresses this by balancing relevance with novelty or aspect coverage, using methods such as Maximal Marginal Relevance (MMR), xQuAD, determinantal point processes, and diversified web ranking approaches (Agrawal et al., 2009; Carbonell and Goldstein, 1998; Santos et al., 2010; Kulesza and Taskar, 2012). Prior work has also shown that the appropriate degree of diversification should depend on query ambiguity rather than being fixed uniformly (Santos et al., 2011).

This dissertation studies *top- k diversified vector search*. Given a query embedding $q \in \mathbb{R}^d$ and a collection of document embeddings $D = \{x_i\}_{i=1}^N$, the goal is to return k results that are both relevant to the query and diverse with respect to one another. The project focuses on a practical two-stage serving scenario: a first-stage retriever produces a candidate pool, and a second-stage reranker selects the final top- k list. The key idea is *per-query adaptivity*: instead of applying one fixed retrieval and reranking policy to every query, the system estimates whether the query is ambiguous or specific and whether temporal freshness matters, then uses these signals to control both stages.

The final system combines three control sources: a learned intent regressor, rule-based factual fallback patterns, and lightweight temporal detection. These signals choose between IVF and HNSW, adjust ANN search breadth and candidate pool size, and set the MMR trade-off parameters for relevance, diversity, and freshness. The reranking stage is implemented with incremental MMR, which requires memory linear in the size of the first-stage candidate pool rather than storing a full quadratic candidate–candidate similarity matrix. The result is a query-aware retrieval pipeline that makes diversified retrieval practical in a realistic ANN serving setting.

1.1 Problem Statement

Let $q \in \mathbb{R}^d$ denote the L_2 -normalised query embedding and let $x \in \mathbb{R}^d$ denote a generic L_2 -normalised document embedding. Under this assumption, cosine similarity reduces to a dot product:

$$\text{sim}(q, x) = \frac{q^\top x}{\|q\| \|x\|} = q^\top x. \quad (1.1)$$

Let the vector database be

$$D = \{x_i\}_{i=1}^N,$$

where $x_i \in \mathbb{R}^d$ is the embedding of the i -th corpus item and N is the corpus size. We write N_c for the size of the first-stage candidate pool returned by the retriever. A relevance-only retriever returns the top- k items with highest $\text{sim}(q, x)$. Diversified retrieval instead seeks a set $S \subseteq D$ of size k that remains close to the query while reducing redundancy among the selected items. A generic objective is

$$\max_{S \subseteq D, |S|=k} \sum_{x \in S} \text{Rel}(q, x) - \alpha \sum_{\substack{x_i, x_j \in S \\ i < j}} \text{Red}(x_i, x_j), \quad (1.2)$$

where Rel denotes query relevance, Red measures redundancy between selected items, and $\alpha \geq 0$ controls the relevance–diversity trade-off.

The challenge is not only to define this objective, but to realise it efficiently. The relevance–diversity trade-off depends on the query: ambiguous or exploratory queries often benefit from broader coverage, while specific or fact-seeking queries should remain close to high-precision retrieval. In addition, real systems usually rely on approximate candidate generation, so reranking must operate on an imperfect ANN pool. Finally, reranking can become expensive when large candidate sets require many pairwise similarity computations. This project therefore addresses both ranking quality and serving practicality.

1.2 Project Overview and Key Idea

The system follows a two-stage retrieval architecture.

1. **Candidate retrieval.** A first-stage retriever generates a candidate pool of size N_c using either exact search or an ANN backend.

2. **Diversified reranking.** A second-stage reranker selects the final top- k results using an MMR-style objective that trades relevance against redundancy.

What distinguishes the project from a fixed diversified retrieval baseline is that both stages are query-adaptive. Before retrieval begins, the system estimates:

- an intent score $s_I \in [0, 1]$, where lower values indicate ambiguous or exploratory queries and higher values indicate specific or fact-seeking queries;
- a temporal score $s_T \in [0, 1]$, where higher values indicate stronger sensitivity to freshness or recency.

These signals are then used to adapt:

- the retrieval backend (IVF or HNSW);
- ANN search parameters such as `nprobe` (the number of IVF clusters probed), `ef_search` (the HNSW beam width), and the candidate pool size N_c ;
- the relevance–diversity trade-off parameter λ in MMR;
- the temporal bonus β used by freshness-aware reranking.

A central design choice is that these control decisions are made *before* candidate retrieval. Unlike score-based adaptive- λ methods, which can only react after a candidate score distribution has been observed, the proposed system can adapt both reranking strength and first-stage retrieval behaviour.

The system is evaluated as a relevance–diversity trade-off rather than a single-metric optimisation problem. Later experiments show that strong fixed baselines remain highly competitive, but the final adaptive configuration behaves in the intended query-sensitive way: it broadens coverage for ambiguous queries and becomes more conservative for specific ones. Its contribution therefore lies less in dominating every baseline on one scalar metric than in combining competitive ranking quality with adaptive backend selection, search-width control, and realistic serving behaviour.

1.3 Contributions

The main contributions are as follows.

1. A query-adaptive two-stage diversified retrieval pipeline that jointly controls ANN candidate generation and reranking.
2. A three-signal controller combining a learned intent model, factual rules, and temporal detection to guide backend selection, search breadth, and reranking strength.
3. An incremental MMR reranker with $O(N_c)$ memory, extended with a relevance floor, a freshness term, and a cluster-aware penalty for ambiguous IVF queries.
4. An empirical evaluation including end-to-end relevance/diversity results, ablations, ANN recall–latency sweeps, and independent intent-model evaluation.

Chapter 2

Background

This chapter reviews the main foundations of diversified vector retrieval: dense vector search, approximate nearest neighbour (ANN) indexing, diversification methods, and the relevance–diversity metrics used later in the report.

2.1 Vector Similarity Search

Dense retrieval represents queries and corpus items as vectors in a shared embedding space. Let $E(\cdot)$ denote a text encoder. Given a query text q_{text} and a document text x_{text} , the encoder produces dense embeddings

$$q = E(q_{\text{text}}), \quad x = E(x_{\text{text}}).$$

In the bi-encoder setting, queries and documents are encoded independently, so corpus embeddings can be pre-computed offline. Ranking then compares the query embedding with stored document vectors using cosine similarity, or equivalently dot product when the embeddings are normalised (Reimers and Gurevych, 2019; Karpukhin et al., 2020).

This project uses a Sentence-Transformers encoder based on MiniLM. Sentence-Transformers produce semantically meaningful sentence embeddings, while MiniLM is a compact distilled transformer with low inference cost (Reimers and Gurevych, 2019; Wang et al., 2020; Sentence-Transformers).

Under the L_2 -normalisation assumption introduced in Section 1.1, cosine similarity reduces to the inner product, so retrieval in this project uses

$$\text{sim}(q, x) = q^\top x.$$

Exact top- k retrieval can therefore be implemented as a matrix–vector multiplication followed by top- k selection. This is a useful baseline, but its cost grows linearly with corpus size N , motivating ANN indexes for large collections.

A cross-encoder scores a query–document pair jointly and can provide stronger fine-grained relevance estimates, but is usually too expensive for first-stage retrieval over large corpora (Nogueira and Cho, 2019). This project therefore focuses on bi-encoder retrieval for candidate generation.

2.2 Approximate Nearest Neighbour Indexes

Approximate nearest neighbour (ANN) search reduces retrieval latency by avoiding an exhaustive scan of the corpus. Instead of returning exact nearest neighbours, an ANN index produces an approximate top- N_c candidate set, where N_c is typically much larger than the final top- k list. A downstream reranker then operates on this smaller candidate pool. The key trade-off is between speed and recall: stronger approximation lowers latency but increases the risk of missing relevant neighbours (Johnson et al., 2017; Jégou et al., 2011; Malkov and Yashunin, 2020).

2.2.1 Inverted File (IVF) Indexes

An inverted file (IVF) index partitions the vector space into a fixed number of coarse regions, typically by clustering corpus vectors with a k -means-style procedure (Arthur and Vassilvitskii, 2007; Jégou et al., 2011). Each vector is assigned to its nearest centroid, producing a set of inverted lists. At query time, the system selects the nearest regions and searches only the vectors in those lists.

The main control parameter is the number of probed clusters, usually denoted by `nprobe`. Larger `nprobe` generally improves recall but increases latency; smaller `nprobe` is faster but risks missing relevant vectors outside the selected coarse cells (Jégou et al., 2011).

In this project, IVF is attractive for ambiguous queries because its coarse partitioning can expose multiple semantic regions and make it easier to retrieve candidates covering different facets of a broad or multi-meaning query.

2.2.2 Graph-Based ANN: HNSW

Hierarchical Navigable Small World (HNSW) graphs organise the corpus as a multi-layer proximity graph (Malkov and Yashunin, 2020). Higher layers are sparse and support long-range navigation, while lower layers are denser and refine search locally. A query starts from an entry point in the upper layers, moves greedily toward the query, and then performs beam search on lower levels to collect high-quality neighbours.

The search quality of HNSW is commonly controlled by the beam width parameter ef_{search} . Larger values usually improve recall at the cost of more computation. In practice, HNSW often provides an excellent recall–latency trade-off for focused nearest-neighbour search and is therefore well suited to specific queries.

Conceptually, IVF and HNSW have different retrieval biases. IVF explores the space by probing coarse regions, while HNSW navigates local neighbourhoods in a proximity graph. This matters later because the system chooses between the two backends adaptively based on predicted query intent.

2.3 Diversification and MMR

For ambiguous or exploratory queries, returning only the most similar items is often not enough. A relevance-only ranker can be highly redundant because nearby points in the

embedding space may correspond to paraphrases or semantically overlapping content. Search-result diversification addresses this by balancing *relevance* to the query with *novelty* relative to already selected items (Agrawal et al., 2009; Carbonell and Goldstein, 1998).

Prior work often treats diversification as coverage over latent subtopics or query aspects. xQuAD explicitly rewards documents that cover aspects not yet satisfied by the current ranked list (Santos et al., 2010). Determinantal point processes (DPPs) model negative correlations directly and prefer diverse subsets (Kulesza and Taskar, 2012). There is also work on intent-aware diversification, where the degree of diversification depends on estimated query ambiguity (Santos et al., 2011).

In this project, the main reranking method is Maximal Marginal Relevance (MMR), originally proposed by Carbonell and Goldstein (1998). Given a candidate pool C and a selected set S , MMR chooses the next item by

$$x^* \leftarrow \arg \max_{x \in C \setminus S} \left(\lambda \text{sim}(q, x) - (1 - \lambda) \max_{y \in S} \text{sim}(x, y) \right),$$

where $\lambda \in [0, 1]$ controls the relevance–diversity trade-off.

MMR fits this project because it is simple, greedy, and easy to deploy in a low-latency pipeline. It does not require explicitly defined subtopics, and its trade-off parameter λ can be adapted on a per-query basis. Compared with xQuAD and DPPs, it is easier to integrate with ANN candidate retrieval and incremental serving-time computation.

2.4 Evaluation Metrics for Relevance and Diversity

Diversified retrieval does not have a single universally correct ranked list. Result quality depends on both relevance and coverage, so offline evaluation usually relies on multiple complementary proxy metrics rather than one ground-truth score (Clarke et al., 2008; Chapelle et al., 2011; Santos et al., 2010, 2011; Kulesza and Taskar, 2012). The metrics used in this dissertation capture candidate recall, final-list relevance, internal diversity, and coarse semantic coverage.

2.4.0.0.1 Recall@k. For candidate retrieval, recall measures how many exact nearest neighbours are recovered by the ANN stage:

$$\text{Recall@}k = \frac{|\text{pred}_k \cap \text{gt}_k|}{|\text{gt}_k|},$$

where pred_k denotes the ANN-retrieved top- k set and gt_k denotes the exact top- k reference set.

2.4.0.0.2 Mean relevance. Given a final result set $S = \{x_1, \dots, x_k\}$, mean relevance measures the average similarity of selected items to the query:

$$\text{MeanRel}(S, q) = \frac{1}{k} \sum_{i=1}^k \text{sim}(q, x_i).$$

2.4.0.0.3 Intra-list diversity (ILD). ILD measures how dissimilar the selected results are from each other:

$$\text{ILD}(S) = 1 - \frac{2}{k(k-1)} \sum_{i < j} \text{sim}(x_i, x_j).$$

Higher ILD indicates lower redundancy within the final ranked list (Ziegler et al., 2005).

2.4.0.0.4 Coverage over semantic clusters. When cluster assignments are available, coverage estimates how many coarse semantic regions are represented:

$$\text{Coverage}@k = \frac{|\{c_{x_i}\}_{i=1}^k|}{k},$$

where c_{x_i} is the cluster label of item x_i .

2.4.0.0.5 Combined relevance–diversity score. To summarise the trade-off in a single scalar quantity, a harmonic-style combination can be used:

$$F1_{\text{RD}} = \frac{2 \cdot \text{MeanRel} \cdot \text{ILD}}{\text{MeanRel} + \text{ILD}}.$$

Although no single combined score is sufficient on its own, it is useful for comparing systems that must balance both objectives.

Taken together, these metrics reflect the central tension of diversified retrieval: a list can be highly relevant but redundant, or diverse but unfocused. The experimental chapter therefore evaluates systems using multiple metrics rather than relying on a single number.

Chapter 3

System Design

This chapter describes the query-adaptive retrieval pipeline used in the final system. Its core design choice is to make control decisions *before* reranking: the system first estimates query type, then uses this information to choose a retrieval backend, adjust search breadth, and set the relevance–diversity trade-off (Santos et al., 2011; Jégou et al., 2011; Malkov and Yashunin, 2020; Carbonell and Goldstein, 1998).

3.1 System Overview

The system has two phases.

Offline preparation. The corpus is encoded into dense L_2 -normalised vectors using a Sentence-Transformers encoder based on MiniLM (Reimers and Gurevych, 2019; Wang et al., 2020). The ANN backends are then built: IVF clusters the corpus into coarse partitions (Arthur and Vassilvitskii, 2007; Jégou et al., 2011), while HNSW constructs a hierarchical proximity graph (Malkov and Yashunin, 2020). In parallel, the intent model is trained on synthetically labelled queries and saved with a fitted scaler and an exact-match label cache.

Online serving. At query time, the pipeline is:

query text $\rightarrow$ sentence embedding $\rightarrow$ query analysis $\rightarrow$ index selection $\rightarrow$ adaptive top- N_c retrieval $\rightarrow$ adaptive MMR reranking $\rightarrow$ final top- k .

The online path is therefore a two-stage pipeline: candidate generation followed by diversity-aware reranking. Unlike a fixed retrieval baseline, the first stage is also adaptive: the query-analysis module decides whether to explore broadly across multiple semantic regions or stay close to a high-relevance nearest-neighbour regime (Santos et al., 2011).

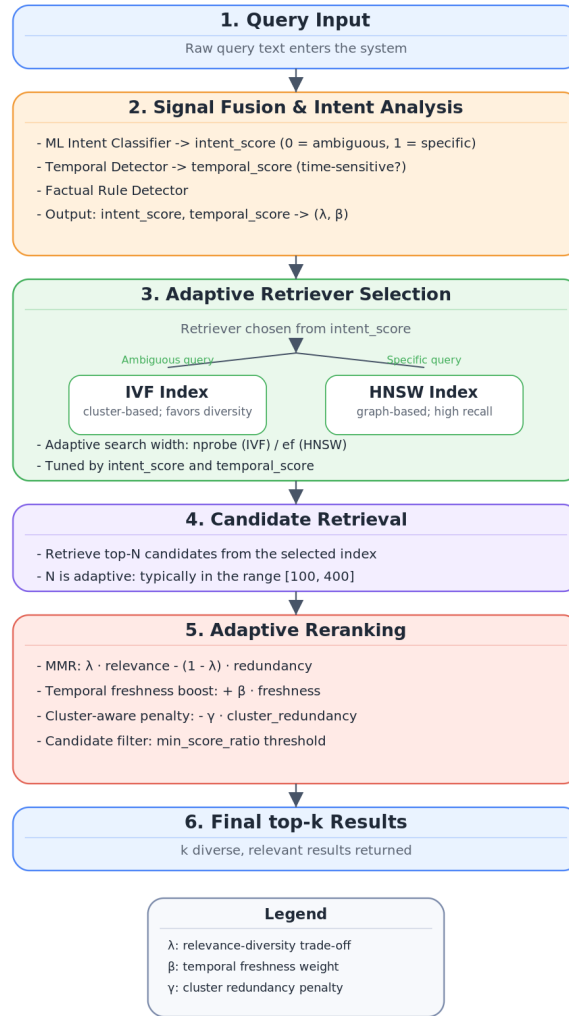


Figure 3.1: Query-adaptive diversified vector retrieval pipeline. The controller combines a learned intent model with temporal and factual rules, chooses between IVF and HNSW, adapts ANN search breadth and candidate-pool size, and then applies reranking with relevance, diversity, freshness, and optional cluster-aware penalties.

3.2 Three-Signal Query Analysis

The system computes two scalar control signals for each query:

- **intent score** $s_I \in [0, 1]$, where lower values indicate ambiguous or exploratory queries and higher values indicate specific, well-constrained queries;
- **temporal score** $s_T \in [0, 1]$, where higher values indicate stronger recency sensitivity.

These signals are produced by combining a learned intent regressor with two lightweight rule-based components. The learned model provides semantic generalisation, while the rules stabilise short fact-seeking and time-sensitive queries where pure embedding-based

prediction can be brittle.

3.2.1 Learned intent model

The first control signal is produced by a learned intent regressor. Its output is a scalar score $s_I^{\text{ML}} \in [0, 1]$, where values close to 0 indicate ambiguous or exploratory queries and values close to 1 indicate specific or fact-seeking queries. A regression model is used instead of hard classification because query specificity is better viewed as a continuum than as a binary label.

The model is trained on a merged dataset of synthetically labelled queries generated with ChatGPT. After deduplication, the dataset contains 5,000 examples. Each training instance is a triple (q_{text}, y, r) , where q_{text} is the raw query text, $y \in [0, 1]$ is the target specificity score, and r is a short rationale retained only for later manual inspection of label quality. The dataset mixes short ambiguous keywords, natural-language search requests, and more specific product- or fact-oriented queries.

Given an input query q_{text} , the intent score is computed in four steps:

$$\begin{aligned} z &= E(q_{\text{text}}), & z &\in \mathbb{R}^{384}, \\ \tilde{z} &= g(z), \\ \hat{y} &= f(\tilde{z}), \\ s_I^{\text{ML}} &= \text{clip}(\hat{y}, 0, 1). \end{aligned} \tag{3.1}$$

Here, $E(\cdot)$ denotes the pre-trained all-MiniLM-L6-v2 sentence encoder, which maps the query text to a 384-dimensional embedding vector (Reimers and Gurevych, 2019; Wang et al., 2020; Sentence-Transformers). The function $g(\cdot)$ is the saved `StandardScaler` transform, which standardises each feature of the embedding using the mean and standard deviation estimated on the training set. The function $f(\cdot)$ is the learned regression model. Its raw output is denoted by $\hat{y}$. Finally, $\text{clip}(\hat{y}, 0, 1)$ forces the prediction to stay in the valid range $[0, 1]$:

$$\text{clip}(x, 0, 1) = \begin{cases} 0, & x < 0, \\ x, & 0 \leq x \leq 1, \\ 1, & x > 1. \end{cases}$$

In words, the model first embeds the query, then standardises the embedding, predicts a raw specificity score, and finally truncates that score to the interval $[0, 1]$.

Model selection is performed on an internal 80/20 train–test split. The implementation compares three regressors: (i) an MLP regressor with hidden layers (256, 128, 64), `max_iter=1500`, and early stopping; (ii) a random forest regressor with 200 trees and `max_depth=15`; and (iii) a gradient boosting regressor with 200 estimators and `max_depth=6`. The deployed model is the gradient boosting regressor, which performs best on the internal split. Independent generalisation results are reported later in Section 7.6.

At inference time, the module first checks an exact-match JSON cache of previously labelled queries. If the query is found in the cache, the stored score is returned directly. Otherwise, the system runs the pipeline in Eq. 3.1: it embeds the query, standardises the embedding, applies the regressor, and clips the output to $[0, 1]$. If neither cache nor model prediction is available, the system falls back to a neutral default score of 0.5.

Because this score is available *before* candidate retrieval, it can be used to control back-end selection, ANN search breadth, candidate-pool size, and the reranking parameter λ .

For intuition about the scale, low scores are assigned to ambiguous queries such as *spring* and *mercury*, whereas high scores are assigned to specific queries such as *Fall of Berlin – 1945* and *Who is the US president in 2018*. These examples illustrate the meaning of the scale, although the final controller also includes factual fallback rules beyond the learned regressor itself.

Because the supervision is synthetic rather than collected from real user behaviour, these labels should be understood as a practical training signal rather than as ground-truth user intent. Some label noise is therefore likely, especially near the soft boundary between exploratory and specific queries, but the regressor remains useful as a pre-retrieval control signal.

3.2.2 Factual fallback rules

A second component provides a rule-based fallback for factoid queries. These are often short and syntactically simple, which can make learned ambiguity prediction less stable. The detector looks for several classes of factual patterns:

- identity and definition forms, such as *who is*, *what is*, and *what does X mean*;
- event and time forms, such as *when did*, *when was*, and *what year*;
- quantity and measurement forms, such as *how many*, *how much*, and *population of*;
- role or office queries, such as *who is president of* or *who is the CEO of*.

These patterns cover cases where a short query still has a narrow information need. The factual detector therefore produces a separate rule-based specificity score $s_F \in [0, 1]$, where larger values indicate that the query matches a clearer fact-seeking pattern. This score is computed independently of the learned regressor and acts as a fallback when short factual queries are underestimated by the learned model. If the rule-based score is larger than the learned intent score, the system uses the higher value:

$$s_I \leftarrow \max(s_I^{\text{ML}}, s_F). \quad (3.2)$$

3.2.3 Temporal detection

A third component detects recency-sensitive information needs. Unlike ambiguity, temporal sensitivity is captured effectively by lexical and pattern-based cues. The detector looks for three types of evidence.

Strong temporal signals. These include terms such as *now*, *current*, *latest*, *today*, explicit years, and role-based queries whose answers may change over time.

Moderate temporal signals. These include terms such as *price*, *cost*, *salary*, *release date*, *trending*, and *new*.

Aggregation. When multiple signals are present, the detector takes the maximum matched strength and adds a small bonus for repeated evidence:

$$s_T = \text{clip}(\max(\text{signals}) + \min(0.1, 0.03 \cdot (n - 1)), 0, 1), \quad (3.3)$$

where n is the number of matched temporal cues.

3.2.4 Fusion into control parameters

The three signals are fused into the parameters used by retrieval and reranking:

1. predict a base intent score s_I^{ML} with the learned regressor;
2. if the factual detector suggests a clearer specific interpretation, override the intent score using Eq. 3.2;
3. compute the temporal score s_T from lexical and pattern-based cues;
4. map the final s_I and s_T to retrieval and reranking parameters.

The relevance–diversity trade-off parameter is

$$\lambda_0 = \lambda_{\min} + s_I(\lambda_{\max} - \lambda_{\min}), \quad \lambda_{\min} = 0.30, \lambda_{\max} = 0.90. \quad (3.4)$$

Ambiguous queries therefore receive a lower λ , while specific queries receive a higher one.

Temporal sensitivity controls an additional freshness bonus:

$$\beta = \beta_{\min} + s_T(\beta_{\max} - \beta_{\min}), \quad \beta_{\min} = 0, \beta_{\max} = 0.15. \quad (3.5)$$

For strongly temporal queries, the system also nudges λ upward before reranking:

$$\lambda = \begin{cases} \text{clip}(\lambda_0 + 0.1 s_T, 0.30, 0.90), & s_T > 0.5, \\ \lambda_0, & \text{otherwise.} \end{cases} \quad (3.6)$$

This prevents highly time-sensitive queries from becoming over-diversified.

The query analysis module therefore outputs the control state

$$(s_I, s_T, \lambda, \beta, e),$$

where the exploration score e is defined in the next section.

Table 3.1: Main query-time control variables.

Symbol	Meaning	Typical role
s_I	intent score	ambiguity / specificity estimate
s_T	temporal score	recency sensitivity estimate
λ	MMR trade-off	relevance versus diversity
β	freshness weight	temporal reranking bonus
e	exploration score	ANN search breadth control
N_c	candidate pool size	number of retrieved candidates
$nprobe$	IVF search width	number of coarse clusters searched
ef_{search}	HNSW beam width	graph-search breadth

3.3 Dynamic Index Selection

The retrieval backend is chosen directly from the final intent score:

$$s_I < 0.4 \Rightarrow \text{IVF}, \quad s_I \geq 0.4 \Rightarrow \text{HNSW}. \quad (3.7)$$

The threshold 0.4 is a practical development heuristic rather than a theoretically derived optimum. It reflects the intended separation between broad exploratory queries and more specific ones. IVF exposes multiple coarse semantic regions through its clustered structure (Arthur and Vassilvitskii, 2007; Jégou et al., 2011), which makes it useful for ambiguous queries that may require facet coverage. HNSW tends to produce stronger nearest-neighbour recall with less semantic drift (Malkov and Yashunin, 2020), making it better suited to specific queries. This dissertation therefore treats 0.4 as a deployment heuristic rather than an experimentally validated optimum; a dedicated sensitivity study is left to future work.

3.4 Adaptive ANN Search Parameters

Once the backend has been chosen, the system adjusts search breadth using an exploration score $e \in [0, 1]$. This controls $nprobe$ for IVF and ef_{search} for HNSW (Jégou et al., 2011; Malkov and Yashunin, 2020). In the current system, e is derived from the intent score through a piecewise-linear mapping:

$$e_0(s_I) = \begin{cases} 1.0 - 1.33 s_I, & s_I \in [0.0, 0.3), \\ 0.6 - 0.75 (s_I - 0.3), & s_I \in [0.3, 0.7), \\ 0.3 - 1.0 (s_I - 0.7), & s_I \in [0.7, 1.0]. \end{cases} \quad (3.8)$$

Here $e_0(s_I)$ denotes the base exploration score before temporal adjustment.

This design creates three regimes: strong exploration for low-intent queries, smoother changes in the mid-range, and conservative behaviour for high-intent queries.

Temporal sensitivity widens exploration slightly:

$$e = \text{clip}(e_0(s_I) + 0.15 s_T, 0, 1). \quad (3.9)$$

The final exploration score is mapped to the index parameters:

- IVF: $nprobe \in [8, 48]$,
- candidate pool size: $N_c \in [100, 400]$,
- HNSW: $ef_{search} \in [32, 200]$.

A simple linear mapping is used:

$$\text{map}(e; a, b) = a + e(b - a). \quad (3.10)$$

Table 3.2: Adaptive ANN parameters controlled by the exploration score.

Parameter	Range	Role
IVF $nprobe$	$[8, 48]$	number of coarse clusters searched
Candidate pool N_c	$[100, 400]$	number of candidates passed to reranking
HNSW ef_{search}	$[32, 200]$	beam width during graph search

Low-intent queries therefore receive broader exploration and larger candidate pools, while high-intent queries stay closer to a relevance-first nearest-neighbour regime. When IVF is selected, the system can additionally enable cluster-balanced candidate sampling to encourage broader semantic coverage before reranking.

Overall, query analysis first estimates *what kind of query this is*; those signals then decide *how broadly to retrieve*; and only after that does the reranker decide *which relevance–diversity trade-off to apply within the candidate pool*. This separation allows the system to adapt both retrieval and reranking rather than treating diversification as a purely final-stage operation.

Chapter 4

Candidate Retrieval Backends

This chapter describes the retrieval backends used for first-stage candidate generation. All three backends implement a common interface,

```
search(query, topN) → (indices, scores),
```

where `topN` is the candidate-pool size N_c supplied to the reranker. The backends considered are an exact brute-force retriever, an IVF index, and an HNSW graph index. Brute-force retrieval serves as an exact reference and evaluation baseline, whereas the adaptive serving pipeline selects between IVF and HNSW according to the controller described in Chapter 3.

4.1 Brute-force retrieval

The brute-force baseline computes the similarity between the query and every corpus vector. Let $q \in \mathbb{R}^d$ be a query embedding and $X \in \mathbb{R}^{N \times d}$ the matrix of corpus embeddings. The similarity scores are

$$s = Xq. \tag{4.1}$$

Since all vectors are normalised, this is equivalent to cosine similarity.

The top- N_c results are extracted from the score vector. In practice, the implementation uses partial selection (for example, `argpartition`) followed by sorting only the top block, rather than sorting all N items.

Brute-force retrieval serves two roles. First, it is a relevance-only baseline with 100% recall. Second, it provides the exact nearest-neighbour reference used to evaluate ANN recall. Its main limitation is computational cost: each query requires a full scan of the corpus, which is $O(Nd)$.

4.2 IVF index

The IVF backend is a pure NumPy implementation of an inverted file index. It partitions the vector space into a fixed number of coarse clusters and searches only a subset

of them for each query. This reduces the number of vector comparisons relative to brute-force search, at the cost of possible recall loss if relevant items lie outside the probed clusters.

4.2.1 Training

The corpus is partitioned into $nlist = 128$ clusters using a NumPy implementation of k -means with k -means++ initialisation (Arthur and Vassilvitskii, 2007). After initialisation, the centroids are refined for 20 iterations by repeatedly assigning corpus vectors to their nearest centroid, recomputing centroid means, and re-normalising the centroids so that later similarity calculations remain consistent with cosine-style dot-product retrieval.

Once training is complete, each corpus vector is assigned a cluster id $c_i \in \{1, \dots, nlist\}$ and the index stores:

- an `assignments` array containing the cluster id of each corpus vector;
- a set of inverted lists, each containing the ids of the vectors assigned to one centroid.

The `assignments` array is also reused later by cluster-aware reranking.

4.2.2 Search

At query time, the system first compares the query vector against all coarse centroids and selects the top- $nprobe$ clusters. Only the vectors in the corresponding inverted lists are then scored against the query. A rough complexity model is

$$O\left(\frac{nprobe}{nlist}Nd\right), \quad (4.2)$$

ignoring the relatively small centroid-scoring overhead. The main control parameter is therefore $nprobe$: increasing it improves recall by searching more clusters, while decreasing it reduces latency.

In this project, IVF is especially useful for ambiguous queries because the candidate set is drawn from several coarse semantic regions and can therefore expose multiple possible interpretations before reranking.

4.2.3 Cluster-balanced sampling

For ambiguous queries, the IVF retriever can operate in a cluster-balanced mode. Instead of taking the global top- N_c candidates from all probed clusters, the retriever first allocates an approximate quota to each searched cluster:

$$\text{quota} = \left\lceil \frac{N_c}{nprobe} \right\rceil. \quad (4.3)$$

Within each probed cluster, items are ranked by query similarity and up to `quota` items are retained. The per-cluster selections are then merged into a single candidate pool.

If some clusters contain fewer than the allocated quota, the remaining capacity is filled from leftover candidates in other probed clusters according to query similarity. This keeps the final candidate pool close to N_c while encouraging broader semantic coverage than a purely global top- N_c selection. The purpose is not to maximise strict first-stage relevance, but to provide a more diverse candidate pool for later reranking.

4.3 HNSW index

The second ANN backend is a pure NumPy implementation of Hierarchical Navigable Small World graphs (HNSW) (Malkov and Yashunin, 2020). HNSW stores the corpus as a multi-layer proximity graph in which upper layers are sparse and lower layers are dense. Every item appears in layer 0, while only a subset appears at higher layers. The upper layers support long-range movement and the lower layers refine the search locally.

4.3.1 Construction

Each node is assigned a maximum level according to a geometric distribution:

$$\ell = \lfloor -\log(u) m_L \rfloor, \quad m_L = \frac{1}{\log M}, \quad u \sim \text{Uniform}(0, 1). \quad (4.4)$$

This makes most nodes appear only in layer 0 and progressively fewer appear at higher layers.

The implementation uses

$$M = 16, \quad M_{\max 0} = 32, \quad ef_{\text{construction}} = 200.$$

Here M is the degree limit on upper layers, $M_{\max 0}$ is the larger limit used on layer 0, and $ef_{\text{construction}}$ controls beam width during construction.

Insertion proceeds in two stages. First, the new node descends greedily through the upper layers to find a good entry point. Second, from the target layer down to layer 0, the algorithm performs beam search with width $ef_{\text{construction}}$ to collect candidate neighbours, then keeps the best ones and links them bidirectionally. When a node exceeds the degree cap, weaker edges are pruned.

4.3.2 Search

At query time, HNSW also uses two stages. The search first descends greedily through the upper layers to reach a good entry point near the query. It then runs beam search on layer 0 with width ef_{search} , maintaining:

- a candidate set of nodes that may still be explored;
- a result set containing the current best neighbours found so far.

The search stops when the best unexplored candidate is already worse than the worst item in the current result set, provided that the result set has reached the target beam width. A practical complexity model is

$$O(\log N \cdot ef_{\text{search}} \cdot M \cdot d). \quad (4.5)$$

Compared with IVF, HNSW is typically better aligned with strict nearest-neighbour retrieval. It is therefore the preferred backend for specific queries in the adaptive pipeline, where preserving relevance is more important than broad cluster-level exploration.

4.4 Summary

The three backends play complementary roles. Brute-force retrieval provides an exact reference and a strong relevance-only baseline. IVF provides fast, cluster-structured candidate generation and is especially useful when broader semantic coverage is desired. HNSW provides a graph-based high-recall ANN alternative that stays closer to strict nearest-neighbour search and is therefore well suited to specific queries. The next chapter builds on these candidate generators by describing how the final top- k list is produced through diversified reranking.

Chapter 5

Diversified Re-ranking

This chapter describes the second stage of the system: diversified reranking over the candidate pool returned by the retrieval backends. The core objective is to preserve query relevance while reducing redundancy among the selected results. The main reranking mechanism is Maximal Marginal Relevance (MMR), extended with practical controls for relevance filtering, temporal freshness, and cluster-aware coverage (Carbonell and Goldstein, 1998).

5.1 Standard MMR

As introduced in Section 2.3, the reranking stage uses the classical maximal marginal relevance (MMR) objective. We restate it here in the notation used by the later extensions. Given a candidate set C of size N_c , let S denote the set of items already selected; the final ranking is given by the order in which items are added to S . At each step, MMR chooses

$$x^* \leftarrow \arg \max_{x_i \in C \setminus S} (\lambda \text{sim}(q, x_i) - (1 - \lambda) \text{Red}(x_i; S)), \quad (5.1)$$

where $\lambda \in [0, 1]$ controls the relevance–diversity trade-off (Carbonell and Goldstein, 1998).

In the classical max-redundancy formulation,

$$\text{Red}(x_i; S) = \max_{y \in S} \text{sim}(x_i, y), \quad (5.2)$$

with the convention that $\text{Red}(x_i; \emptyset) = 0$. Thus, the first selected item is simply the most relevant candidate, and each later step penalises a candidate according to its strongest overlap with the current list.

Equation 5.1 is applied repeatedly until $|S| = k$. This greedy structure is attractive in a serving-time pipeline because it is simple, interpretable, and easy to extend. In the present system, the base MMR objective is later augmented with a relevance floor, optional temporal freshness, optional cluster penalties, and a quality-aware fallback toward relevance.

5.2 Incremental MMR for efficiency

A straightforward implementation of MMR precomputes the similarity between every pair of candidates, producing an $N_c \times N_c$ candidate–candidate similarity matrix. This requires $O(N_c^2)$ memory, which becomes undesirable when the candidate pool is large. In practice, however, MMR does not need the full matrix at once. At each step, it only needs to know, for each remaining candidate, how redundant that candidate is with respect to the items that have already been selected. The implementation therefore uses an incremental variant that stores only this current redundancy information.

Let

$$X = [x_1^\top; \dots; x_{N_c}^\top] \in \mathbb{R}^{N_c \times d}$$

denote the matrix of candidate vectors, and let S denote the set of items that have already been selected into the output list. Under the classical max-redundancy formulation, the reranker maintains a vector $m \in \mathbb{R}^{N_c}$ whose i -th entry is

$$m_i = \max_{y \in S} \text{sim}(x_i, y). \quad (5.3)$$

In other words, m_i records how similar candidate x_i is to its most redundant already-selected neighbour. When reranking begins, no items have yet been selected, so $S = \emptyset$; by convention, all redundancy values are initialised to zero:

$$m_i = 0 \quad \text{for all } i.$$

With all redundancy values equal to zero, the first MMR step reduces to selecting the most relevant candidate.

Suppose the reranker has just selected a new item x_j . To update the redundancy state, it is sufficient to compute the similarity between x_j and every candidate once. Since the vectors are L_2 -normalised, the vector

$$Xx_j$$

contains exactly these similarities, with i -th entry $\text{sim}(x_i, x_j)$. The redundancy state can therefore be updated elementwise as

$$m \leftarrow \max(m, Xx_j). \quad (5.4)$$

After this update, each candidate keeps the largest similarity it has seen so far to any item in S . The reranker therefore maintains only the information needed for the next greedy decision, rather than materialising the full $N_c \times N_c$ matrix in advance.

If the final ranked list has length k , the reranker performs one such candidate–selected similarity sweep per step. The resulting complexity is therefore $O(kN_c d)$ time and only $O(N_c)$ extra memory.

The implementation also supports an optional mean-redundancy variant. In this case, instead of storing only the maximum similarity to the selected set, the reranker stores a cumulative similarity vector $c \in \mathbb{R}^{N_c}$, where

$$c_i = \sum_{y \in S} \text{sim}(x_i, y).$$

Initially $c_i = 0$ for all candidates, and after selecting a new item x_j the state is updated by

$$c \leftarrow c + Xx_j.$$

The redundancy term is then computed as

$$\text{Red}_{\text{mean}}(x_i; S) = \frac{1}{|S|} \sum_{y \in S} \text{sim}(x_i, y) = \frac{c_i}{|S|}. \quad (5.5)$$

The main pipeline uses the classical max-redundancy version by default, because it penalises near-duplicates more directly, while the mean-redundancy option is retained as a smoother alternative when softer redundancy estimates are desired.

5.3 Relevance floor

Diversification is useful only when applied to candidates that are at least reasonably relevant to the query. Before reranking, the implementation can remove candidates whose query similarity is too far below the best candidate. If s_{max} is the top candidate score, the relevance floor is

$$\tau = s_{\text{max}} \cdot r, \quad r = 0.60 + 0.25 s_I. \quad (5.6)$$

Here $r \in [0.60, 0.85]$ is the relevance-floor ratio. Specific queries therefore use a stricter relevance threshold than ambiguous queries: when the intent is clear, the system filters more aggressively to avoid semantic drift.

In implementation terms, this filter is applied only if enough candidates remain to fill the final top- k list. This avoids pathological cases where over-filtering would leave too few candidates for reranking.

5.4 Cluster-aware penalty

When ambiguous queries are served by IVF, the reranker can exploit the cluster assignments already produced by the index. Let c_i denote the cluster id of candidate x_i . If the selected set already contains several representatives from the same cluster, the candidate receives an additional penalty:

$$\text{ClusterPenalty}(x_i; S) = \gamma \cdot \frac{|\{y \in S : c_y = c_i\}|}{|S|}. \quad (5.7)$$

Here $\gamma \geq 0$ controls the strength of the cluster-level penalty.

This term discourages one cluster from monopolising the final ranked list and is almost free computationally because the cluster ids are already available from IVF. In the deployed serving pipeline, this penalty is enabled only for ambiguous queries routed through the IVF path. Concretely, the current demo uses

$$\gamma = 0.15 \quad \text{for ambiguous IVF queries,} \quad \gamma = 0 \quad \text{otherwise.}$$

As a result, specific queries served by HNSW remain purely relevance-focused and do not pay an extra cluster-level diversity penalty.

5.5 Temporal MMR

For time-sensitive queries, the reranker augments MMR with a freshness term:

$$\text{score}(x_i) = \lambda \text{Rel}(q, x_i) - (1 - \lambda) \text{Red}(x_i; S) + \beta \text{Fresh}(x_i) - \text{ClusterPenalty}(x_i; S). \quad (5.8)$$

Here $\text{Rel}(q, x_i) = \text{sim}(q, x_i)$ is the relevance term and $\text{Fresh}(x_i) \in [0, 1]$ is the freshness prior of candidate x_i .

The first selected item is handled slightly differently, because redundancy is undefined when $S = \emptyset$. In the temporal reranker, the first item is chosen using

$$x^{(1)} \leftarrow \arg \max_{x_i \in C} (\lambda \text{Rel}(q, x_i) + \beta \text{Fresh}(x_i)). \quad (5.9)$$

After the first selection, the standard temporal MMR score in Eq. 5.8 is used for all remaining positions.

The current implementation uses a lightweight source-level freshness prior rather than document timestamps. In the demo configuration, passages from sources with stronger recency expectations can be assigned larger freshness scores than older or more static sources; for example, an earlier version of the system used values such as Wikipedia = 0.7 and MS MARCO = 0.3 as a simple prior. This is an engineering simplification rather than a full temporal model, but it is sufficient to demonstrate how temporal sensitivity can influence reranking. If no freshness array is available, or if $\beta \leq 0$, the implementation falls back directly to the non-temporal incremental MMR routine with zero additional overhead.

5.6 Quality-aware graceful degradation

If the candidate set is weak, strong diversification can pull clearly irrelevant items into the final list. To mitigate this, the system increases λ when the best candidate score is low:

$$\text{if } s_{\max} < 0.5, \quad \lambda \leftarrow \min(0.9, \lambda + (0.5 - s_{\max})). \quad (5.10)$$

This causes the reranker to fall back toward relevance when corpus coverage is poor. Operationally, this step is applied after candidate retrieval but before final reranking. It therefore acts as a safeguard against over-diversification when the first-stage retriever has already produced a weak candidate pool.

5.7 Score-based adaptive- λ baseline

In addition to the final intent-conditioned control policy, the project includes one score-based adaptive- λ baseline, used as L1 in the experimental comparison reported later in Chapter 7. This baseline infers the relevance–diversity trade-off directly from the observed candidate score distribution, without a separate intent model. Earlier development explored simpler gap-based and entropy-based variants, but the main evaluation retains only a single hybrid baseline in order to keep the comparison aligned with the

staged ablation reported later. Since the score distribution is only available after candidate generation, this baseline can adjust reranking strength but cannot control backend selection, ANN search breadth, or candidate-pool size. Although related in spirit to query-performance prediction and score-distribution analysis (Cronen-Townsend et al., 2002), the present method is a lightweight engineering heuristic rather than a direct implementation of classical clarity estimation.

Let the candidate relevance scores be sorted as

$$s_1 \geq s_2 \geq \dots \geq s_{N_c}. \quad (5.11)$$

The baseline uses a hybrid specificity proxy derived from three normalised score statistics.

The first component is the top- k gap

$$g = s_1 - s_k, \quad (5.12)$$

which is mapped to $[0, 1]$ by

$$g_c = \text{clip}\left(\frac{g - g_{\text{low}}}{g_{\text{high}} - g_{\text{low}}}, 0, 1\right). \quad (5.13)$$

A larger gap suggests a clearer dominant interpretation and therefore a larger λ .

The second component is the coefficient of variation of the candidate scores:

$$cv = \frac{\sigma_s}{\bar{s} + \epsilon}, \quad (5.14)$$

where $\bar{s}$ and σ_s denote the mean and standard deviation of the candidate scores, and $\epsilon > 0$ is a small constant for numerical stability. Its normalised form is

$$cv_b = \text{clip}\left(\frac{cv - cv_{\text{low}}}{cv_{\text{high}} - cv_{\text{low}}}, 0, 1\right). \quad (5.15)$$

Higher variation indicates a sharper separation between stronger and weaker candidates, which again supports a more relevance-focused reranking regime.

The third component is the leading-score term

$$s_{b1} = \text{clip}\left(\frac{s_1 - s_{\text{low}}}{s_{\text{high}} - s_{\text{low}}}, 0, 1\right), \quad (5.16)$$

which captures the absolute strength of the best retrieved candidate.

These three cues are combined into a single hybrid score statistic:

$$z = 0.5 g_c + 0.3 cv_b + 0.2 s_{b1}. \quad (5.17)$$

The resulting value is then mapped to the same MMR operating range used elsewhere in the system:

$$\lambda = \lambda_{\min} + z(\lambda_{\max} - \lambda_{\min}), \quad \lambda_{\min} = 0.30, \lambda_{\max} = 0.90. \quad (5.18)$$

Large values of z correspond to a more concentrated candidate distribution and therefore produce a larger λ , whereas flatter score profiles lead to stronger diversification through a smaller λ .

To reduce corpus-specific brittleness, the normalisation bounds

$$g_{\text{low}}, g_{\text{high}}, c_{\text{Vlow}}, c_{\text{Vhigh}}, s_{\text{low}}, s_{\text{high}}$$

are estimated empirically from a held-out query sample rather than fixed as universal constants. In this way, the baseline adapts to the scale of the observed score distribution while remaining simple to compute.

This score-based baseline is intentionally narrower than the final system. It reacts only after a candidate pool has already been retrieved, whereas the proposed intent-conditioned controller predicts query properties before retrieval and uses them to influence backend selection, ANN search parameters, candidate-pool size, and reranking strength. The comparison is therefore methodologically useful: it separates adapting λ from observed candidate scores alone from adapting the full retrieval pipeline through pre-retrieval query analysis.

5.8 Complexity summary

Table 5.1 summarises the main query-time and storage costs of the retrieval and reranking components. The distinction between persistent index storage and extra per-query memory is useful because the ANN backends incur one-off storage costs, whereas reranking mainly uses temporary buffers.

Component	Time per query	Extra query memory	Persistent storage
Brute-force search	$O(Nd)$	$O(N)$ scores	$O(Nd)$ corpus matrix
IVF search	$O\left(\frac{n_{\text{probe}}}{n_{\text{list}}}Nd\right)$ (approx.)	$O(N_c)$	corpus vectors, centroids, and inverted lists
HNSW search	$O(\log N \cdot ef_{\text{search}} \cdot M \cdot d)$ (approx.)	$O(ef_{\text{search}})$ frontier	corpus vectors and graph edges
Incremental MMR	$O(kN_c d)$	$O(N_c)$	none beyond candidates

Table 5.1: Main query-time and storage costs.

Chapter 6

Implementation

This chapter maps the system design onto the actual codebase. The main goal of the implementation is to keep retrieval, query analysis, reranking, and evaluation modular so that the same core components can be reused across both experiments and the demo.

6.1 Codebase structure

The codebase has three layers: `src/diverse_search/` for reusable core modules, `scripts/` for training and evaluation entry points, and `demo/` for the FastAPI application. Within the core library, `index.py`, `brute_force.py`, `ivf.py`, and `hnsf.py` implement retrieval; `mmr.py`, `diversify.py`, and `adaptive_lambda.py` implement reranking and comparison baselines; `intent_model.py` and `temporal.py` implement query analysis; and helper modules such as `metrics.py`, `cluster.py`, `datasets.py`, `experiment.py`, and `search_api.py` support reusable programmatic access and evaluation. The deployed path is therefore simple: `demo/search_api.py` calls the retriever factory, the query-analysis modules, the reranker, and the diagnostic routines, while the same core modules are also reused by the experiment scripts.

6.2 Intent model implementation

The deployed intent module loads its artefacts from `models/intent_v3/`: the saved regressor plus fitted `StandardScaler` in `intent_model.pkl`, and the exact-match cache in `intent_labels.json`. At inference time, `IntentClassifier.predict()` first checks the cache; on a miss, it embeds the query with `all-MiniLM-L6-v2`, applies the stored scaler and regressor, clips the prediction to $[0, 1]$, and falls back to 0.5 if the artefacts are unavailable.

The training script `scripts/train_diverse_intent.py` merges labelled files, removes duplicates, embeds the queries, standardises the 384-dimensional vectors, and compares three lightweight regressors on an internal 80/20 split. Gradient boosting performs best on that split ($R^2 = 0.6964$, $MAE = 0.0934$, $RMSE = 0.1472$), narrowly ahead of random forest, and is therefore used as the deployed checkpoint. This design

is deliberately lightweight: most semantic generalisation comes from the sentence encoder, while the downstream regressor only learns the mapping from query semantics to a specificity score.

6.3 Serving pipeline in the demo

The demo backend in `demo/search_api.py` loads passages, precomputed embeddings, optional source labels and freshness scores, the trained intent model, and the sentence-transformer encoder, then builds brute-force, IVF, and HNSW backends over the normalised corpus embeddings. For each query, the backend (1) encodes the query and runs `query_aware_lambda_and_beta()` to obtain λ , s_I , s_T , and β ; (2) chooses IVF when $s_I < 0.4$ and HNSW otherwise, together with adaptive `topN` and `nprobe/ef_search`; (3) optionally applies the post-retrieval boost

$$\lambda \leftarrow \min(0.9, \lambda + (0.5 - s_{\max}))$$

when the best candidate score is weak, sets

$$\text{min_ratio} = 0.60 + 0.25 s_I,$$

and enables cluster-aware reranking only for ambiguous IVF queries with $\gamma = 0.15$; and (4) calls `mmr_rerank_temporal()` and returns the ranked results together with diagnostics such as mean relevance, diversity, and the F1-style score. The λ boost is intentionally applied after candidate retrieval because it depends on the observed top candidate score.

6.4 Key data structures

Persistent structures are the corpus embedding matrix, passage and metadata arrays, optional freshness scores, IVF centroids with inverted lists and assignment ids, HNSW adjacency lists, and the intent-model artefacts. Query-time state consists mainly of candidate id/score arrays and the incremental MMR redundancy buffer. This separation matches the serving design: large corpus-level structures are built once, while each query allocates only small transient arrays.

Chapter 7

Experiments and Evaluation

This chapter evaluates the proposed system from four complementary angles: (1) overall end-to-end ranking quality, (2) the contribution of individual components through ablation, (3) index-level retrieval behaviour for IVF and HNSW, and (4) qualitative behaviour on representative ambiguous and specific queries.

7.1 Evaluation philosophy and experimental setup

Diversified retrieval does not admit a single universally correct ranked list, because both relevance and diversity depend on query intent and user preference. Accordingly, this project adopts a *proxy-based comparative evaluation* rather than claiming a unique ground-truth optimum for the final top- k list (Clarke et al., 2008; Chapelle et al., 2011). The purpose of the experiments is therefore not to prove that one ranked list is objectively correct, but to assess whether the proposed system yields a more favourable relevance–diversity trade-off than simpler baselines.

The evaluation is organised at two levels. First, candidate-generation quality is assessed using ANN-style vector experiments where exact nearest-neighbour ground truth is available. This makes Recall@ k measurable directly and allows IVF and HNSW to be compared independently of reranking. Second, end-to-end ranking quality is evaluated on the text retrieval corpus used by the demo system. In this setting, the final ranked list is assessed using complementary relevance–diversity proxy metrics rather than a single ground-truth ranking label.

The main end-to-end experiments reported below use the Wikipedia/Wikidata-style passage corpus constructed for this project and used in the demo system, containing approximately 45k passages. The final evaluation set contains 160 queries, partitioned by the deployed controller under the current threshold rule into 76 ambiguous and 84 specific cases. All end-to-end results use the `all-MiniLM-L6-v2` embedding model (Sentence-Transformers), a final list size of $k = 10$, and the same adaptive candidate-pool range $N_c \in [100, 400]$. Any latency numbers reported later should be interpreted comparatively, since all methods were measured in the same runtime environment.

Because the end-to-end evaluation is conducted on a single Wikipedia/Wikidata-style

passage corpus, the reported behaviour should also be interpreted relative to this corpus construction. Passage segmentation, duplicate handling, source filtering, and metadata quality affect the candidate pool seen by the reranker and therefore bound the gains achievable by diversification. In other words, the reranker can reorganise retrieved candidates, but it cannot recover facets that are absent or poorly represented in the underlying corpus.

The main end-to-end comparisons include two simple baselines, a controller-level comparison, and then cumulative extensions of the learned-intent configuration:

- **Relevance-only baseline:** top- k retrieval without reranking;
- **Fixed-MMR baseline:** reranking with a fixed λ ;
- **L1:** score-based adaptive- λ controller baseline;
- **L2:** learned intent controller replacing the score-based adaptive- λ controller;
- **L3:** L2 + dynamic IVF/HNSW backend selection;
- **L4:** L3 + cluster-aware reranking penalty;
- **L5:** L4 + temporal signals and freshness-aware reranking (full adaptive pipeline).

Importantly, L1 is not an earlier stage of the deployed L5 pipeline. It is a heuristic controller baseline used only for the L1–L2 comparison. The cumulative adaptive pipeline begins at L2, and L3–L5 progressively extend that learned-intent configuration.

Unless otherwise noted, all reported values are averaged over queries. Statistical significance is assessed using paired Wilcoxon signed-rank tests over per-query scores (Wilcoxon, 1945), with Bonferroni correction applied to the main family of pairwise comparisons.

7.2 Metrics used in this chapter

The formal definitions of the relevance–diversity metrics are given in Section 2.4. This chapter uses those definitions operationally rather than repeating them in full.

For end-to-end ranking experiments, the main reported quantities are:

- **MeanRel:** average relevance of the final top- k list;
- **ILD:** intra-list diversity of the final top- k list;
- **Avg. F1:** the arithmetic mean of the per-query F1-style relevance–diversity score.

The reported F1 value is computed per query and then averaged across the evaluation set. This is preferable to computing a single corpus-level F1 from aggregate statistics, because it preserves query-level variation and remains compatible with paired significance testing.

For index-level ANN experiments, the main metric is:

- **Recall@ k :** recovery of exact nearest neighbours under approximate search.

When cluster assignments are available, coverage over semantic clusters may also be reported as an auxiliary diagnostic. However, it is interpreted only as a coarse proxy for semantic spread rather than a direct measure of user satisfaction.

7.3 Overall end-to-end quantitative results

Table 7.1 summarises the main end-to-end results on the final query set. The goal of this comparison is not to identify a single universally best ranking, but to examine how different methods trade off relevance and diversity under the same retrieval conditions. The comparison should be read in three parts: two simple baselines, a controller-level comparison between L1 and L2, and then progressive extensions of the learned-intent pipeline from L2 to L5.

Method	MeanRel	ILD	Avg. F1
Relevance-only baseline	0.5495	0.5350	0.5279
Fixed-MMR baseline	0.5245	0.6363	0.5607
L1: score-based controller baseline	0.5431	0.5762	0.5428
L2: learned intent controller	0.4642	0.7163	0.5433
L3: learned intent + dynamic backend selection	0.4565	0.7151	0.5352
L4: learned intent + backend selection + cluster penalty	0.4541	0.7178	0.5337
L5: full learned-intent adaptive pipeline	0.5079	0.6395	0.5504

Table 7.1: Overall end-to-end results on the final query set. L1 is a heuristic controller baseline used only for the L1–L2 comparison; L3–L5 progressively extend the learned-intent configuration introduced in L2.

Several patterns are immediately visible. The relevance-only baseline preserves the highest raw query similarity, as expected, but also yields the lowest diversity. Fixed-MMR is the strongest simple baseline on the current query set, achieving the best overall Avg. F1.

The controller-level comparison between L1 and L2 shows a clear behavioural shift. Replacing the score-based controller with the learned intent controller substantially increases diversity while leaving Avg. F1 almost unchanged. This indicates that the intent model is not merely a small refinement of the heuristic controller; it changes the system’s operating point on the relevance–diversity trade-off.

Within the learned-intent line itself, L5 is the strongest configuration, outperforming L2–L4 on Avg. F1 while remaining substantially more diverse than the relevance-only baseline. These results suggest that the value of the final system does not lie in dominating the strongest simple baseline on a single scalar metric. Instead, its contribution is to combine learned intent control, dynamic backend selection, adaptive search-width control, cluster-aware reranking, and temporal handling in a realistic serving pipeline.

To make the ambiguous/specific distinction explicit, Table 7.2 reports the same core metrics on controller-defined ambiguous and specific subsets.

Query type	Method	MeanRel	ILD	Avg. F1
Ambiguous	Best simple baseline (Fixed-MMR)	0.5290	0.6204	0.5586
Ambiguous	L5 full adaptive pipeline	0.4865	0.6775	0.5531
Specific	Best simple baseline (Fixed-MMR)	0.5205	0.6507	0.5625
Specific	L5 full adaptive pipeline	0.5272	0.6050	0.5480

Table 7.2: Results on controller-defined ambiguous and specific subsets.

These subsets are defined by the deployed controller rather than by external intent annotations, so they should be read as an analysis of system behaviour rather than as a ground-truth intent benchmark. Within that framing, the split is still informative. On ambiguous queries, the full adaptive system behaves as intended: it sacrifices some relevance in exchange for higher diversity. On specific queries, the controller shifts back toward a more relevance-focused regime, raising MeanRel above the best simple baseline but with lower ILD. In both subsets, Fixed-MMR remains the strongest F1 baseline, but the adaptive controller exhibits the intended query-sensitive behaviour.

7.4 Ablation study

The ablation is organised in two steps rather than as a single cumulative stack. First, L1 and L2 compare two alternative mechanisms for controlling the relevance–diversity trade-off: a score-based adaptive- λ heuristic and a learned intent controller. L1 is therefore included only as a controller baseline and is not a constituent stage of the final L5 pipeline. The later stages then progressively extend L2:

- **L1:** score-based adaptive- λ controller baseline;
- **L2:** learned intent controller replacing the score-based controller;
- **L3:** L2 + dynamic IVF/HNSW backend selection;
- **L4:** L3 + cluster-aware reranking penalty;
- **L5:** L4 + temporal signals and freshness-aware reranking.

This structure separates two questions. The L1–L2 comparison isolates the effect of replacing the heuristic λ controller with the learned controller. The later stages then examine how backend selection, cluster-aware reranking, and temporal or freshness-aware controls affect the learned-intent system.

Variant	MeanRel	ILD	Avg. F1
L1: score-based controller baseline	0.5431	0.5762	0.5428
L2: learned intent controller	0.4642	0.7163	0.5433
L3: L2 + dynamic backend selection	0.4565	0.7151	0.5352
L4: L3 + cluster penalty	0.4541	0.7178	0.5337
L5: L4 + temporal signals	0.5079	0.6395	0.5504

Table 7.3: Ablation results. L1 and L2 compare alternative λ -control strategies; L3–L5 then progressively extend the learned-intent configuration.

The ablation reveals two distinct patterns rather than one single monotonic build-up. First, the L1–L2 comparison shows that replacing the score-based controller with the learned intent controller does not improve raw relevance; instead, it substantially increases diversity while leaving Avg. F1 almost unchanged. This is consistent with the intended role of the learned controller: rather than preserving score-based behaviour, it more actively identifies ambiguity and lowers λ when broader semantic coverage is desirable.

Starting from L2, the later stages then refine the learned-intent system. L3 and L4 continue to favour semantic spread, maintaining very high ILD while slightly reducing relevance. L5 partially reverses this trend by recovering relevance and Avg. F1 while reducing ILD. This suggests that the temporal and quality-aware controls help prevent over-diversification when candidate quality is weak or when freshness-sensitive queries should remain more focused.

To support this interpretation, Table 7.4 reports the key paired significance tests for the controller replacement step and selected consecutive extensions of the learned-intent pipeline.

Comparison	Metric	p -value
L2 vs. L1	MeanRel	$< 10^{-6}$
L2 vs. L1	ILD	$< 10^{-6}$
L2 vs. L1	Avg. F1	1.0000
L4 vs. L3	MeanRel	6.5×10^{-5}
L4 vs. L3	ILD	2.5×10^{-3}
L4 vs. L3	Avg. F1	7.3×10^{-5}
L5 vs. L4	MeanRel	$< 10^{-6}$
L5 vs. L4	ILD	$< 10^{-6}$
L5 vs. L4	Avg. F1	$< 10^{-6}$

Table 7.4: Paired significance tests for the controller replacement step and key consecutive learned-intent extensions (Bonferroni-corrected).

These significance tests support the same two-step reading of the ablation. The L2–L1 comparison yields a statistically significant relevance drop and diversity increase, with

no significant change in Avg. F1. This shows that the learned intent controller materially changes the trade-off relative to the heuristic controller rather than merely nudging it.

The later comparisons then quantify refinements within the learned-intent pipeline itself. L4 has a small but statistically reliable effect relative to L3, while L5 significantly improves MeanRel and Avg. F1 relative to L4, confirming that the final stage meaningfully rebalances the system.

7.5 Index-level retrieval experiments

To evaluate the candidate-generation stage independently of reranking, IVF and HNSW are compared on ANN-style vector data where exact nearest-neighbour ground truth is available. The purpose of this experiment is not to study list diversity directly, but to characterise the recall–latency trade-off of the retrieval backends.

In this setting, IVF is evaluated over a sweep of `nprobe` values, while HNSW is evaluated over a sweep of `ef_search` values. Brute-force search provides the exact reference. These results are useful because they show whether the backend distinction used later by the controller is plausible at the retrieval level before any reranking takes place.

Backend	Setting	Recall@k	Latency (ms)
IVF	$nprobe = 8$	0.8268	0.604
IVF	$nprobe = 16$	0.9119	1.142
IVF	$nprobe = 24$	0.9451	1.630
IVF	$nprobe = 32$	0.9620	2.110
IVF	$nprobe = 48$	0.9791	3.491
HNSW	$ef_{search} = 32$	0.9537	1.182
HNSW	$ef_{search} = 50$	0.9537	1.161
HNSW	$ef_{search} = 100$	0.9537	1.155
HNSW	$ef_{search} = 150$	0.9537	1.153
HNSW	$ef_{search} = 200$	0.9537	1.153

Table 7.5: IVF/HNSW recall–latency sweep on the current ANN-style vector benchmark.

The IVF sweep exhibits the expected monotonic trade-off: increasing `nprobe` improves Recall@k at the cost of higher latency. HNSW, by contrast, already attains high recall at the smallest tested beam width, and under the current benchmark its recall and latency change very little over the tested `ef_search` range. A practical interpretation is that the current HNSW implementation saturates early on this benchmark.

These retrieval-level results support the intended division of roles in the deployed system. HNSW behaves like the stronger focused retriever and is therefore well suited to specific queries. IVF is less efficient in raw recall–latency terms on this benchmark, but its coarse cluster structure still provides a useful exploratory substrate for ambiguous queries before reranking. At the same time, these experiments do *not* validate the exact

backend-switch threshold at 0.4. That threshold remains an empirical deployment heuristic in the current system, and a dedicated sensitivity study over alternative switch points is left to future work.

7.6 Intent model evaluation

Since learned intent prediction is a central control signal in the final system, the deployed intent regressor is also evaluated independently on a separate test set of unseen queries. The purpose of this experiment is to assess how well the deployed checkpoint generalises beyond the internal training split used during development.

Metric	Value
MAE	0.1390
RMSE	0.1726
R^2	0.5918
Pearson r	0.8101
Spearman ρ	0.7863

Table 7.6: Independent evaluation of the deployed intent predictor on 60 unseen queries.

These results indicate moderate generalisation. Although the predictor does not recover the reference scores perfectly, the correlation values show that it captures a meaningful ambiguity-to-specificity gradient on previously unseen queries. For the purposes of the final system, this is more important than exact score matching, since the intent score is used primarily as a control signal rather than as a standalone end-user prediction.

To make the outputs easier to interpret at a decision level, the continuous predictions were also grouped into three coarse bands: ambiguous, mid, and specific.

Band	Total	Correct	Accuracy
Ambiguous	13	7	53.8%
Mid	26	14	53.8%
Specific	21	19	90.5%
Overall	60	40	66.7%

Table 7.7: Coarse band accuracy on the independent intent test set.

This breakdown reveals a clear asymmetry. The model is highly reliable on clearly specific queries, but noticeably less stable in the ambiguous and mid ranges. In the confusion pattern, ambiguous queries are usually only shifted upward into the mid band, whereas mid-range queries are more often pushed into the specific band. This suggests that the remaining errors are concentrated near the decision boundaries rather than being uniformly distributed across the scale.

A closer inspection of the largest mismatches indicates a systematic bias rather than purely random noise. Borderline practical or exploratory queries such as *cheap flights*, *plane ticket to Paris*, *gold price prediction*, and *best laptop* are all assigned scores that are noticeably more specific than their reference labels. By contrast, only a small number of clearly specific queries are pushed downward into the mid band. This suggests that the predictor is more sensitive to lexical or topical concreteness than to true retrieval specificity, especially when a query names a concrete topic but still implies an open-ended information need.

This standalone evaluation should therefore be interpreted carefully. First, the supervision remains synthetic rather than directly collected from end users. Second, the intent model is not used in isolation: in the final system it acts as a pre-retrieval controller that influences backend selection and downstream diversification behaviour. The independent test therefore supports a narrower but still useful claim: the intent model provides a moderately informative and directionally correct control signal, even though it is not a fully reliable standalone model of user intent.

7.7 Qualitative case studies

Because diversified retrieval is partly query-dependent and not fully captured by scalar metrics alone, the quantitative results are complemented by qualitative case studies. The examples below compare three settings shown directly in the demo interface: a relevance-only baseline without reranking, a fixed-MMR baseline with $\lambda = 0.7$, and the full adaptive pipeline. To reduce the length of the main text, the corresponding screenshots are moved to Appendix A and are referenced from the individual cases below.

A useful interpretation principle is that diversification can operate at more than one level. Some gains come from covering clearly distinct core senses of a query, whereas other gains come from broader but still interpretable semantic expansion. The cases below therefore distinguish between *useful broadening* and *topical drift*.

Query	Controller behaviour	Main takeaway
<i>spring</i>	IVF, low intent (18%), $\lambda = 0.41$	Useful broadening across core senses and nearby related semantic regions.
<i>mercury</i>	IVF, low intent (20%), $\lambda = 0.42$	Over-diversification: semantic spread grows, but much of it becomes topical drift.
<i>Fall of Berlin – 1945</i>	HNSW, high intent (89%), $\lambda = 0.83$	Conservative reranking keeps the list close to the main historical topic.
<i>Who is the US president in 2018</i>	HNSW, high intent (85%), temporal score 75%, $\lambda = 0.89$, $\beta = 0.112$	Appropriate relevance-first behaviour, though passage retrieval remains imperfect for exact factual answers.

Table 7.8: Summary of the four qualitative case studies.

7.7.1 Ambiguous queries

Case 1: *spring*

The *spring* example is the clearest illustration of useful broadening. The relevance-only baseline is dominated by the seasonal interpretation and contains near-duplicates, whereas fixed MMR broadens the list to several major readings, including a city, a film, and a natural-water sense, giving the strongest trade-off in this case (diversity 69.0, F1 59.9). The full adaptive pipeline increases diversity further to 73.9 and still reaches interpretable alternatives such as a mechanical spring, a hot spring, and nearby spring-related items such as *winter* and *Spring Waltz*, but relevance falls to 48.2. The corresponding screenshot comparison is shown in Appendix A, Figure A.1.

Case 2: *mercury*

The *mercury* example shows the opposite outcome. The baseline already covers several strong interpretations, including the chemical element, the planet, and mythology, and fixed MMR gives the best balance in this case (diversity 68.1, F1 58.9). The full adaptive pipeline pushes diversity to 90.7 but drops relevance to 30.4, introducing much weaker neighbours such as *cox*, *burn*, *Princess Florestine of Monaco*, *collector*, *buffalo*, *sterling*, *ice*, and *Zeus*. This is therefore a genuine failure case: ambiguity-sensitive exploration can overshoot when the candidate pool is noisy or when semantic neighbourhoods are not cleanly separated. The corresponding screenshot comparison is shown in Appendix A, Figure A.2.

7.7.2 Specific queries

Case 3: *Fall of Berlin – 1945*

The full pipeline predicts high query clarity (89%), selects HNSW, and raises λ to 0.83, which is the intended behaviour for a narrow historical query. The relevance-only baseline is already focused, while fixed MMR broadens the ranking and achieves the highest F1 in this example (58.4). The adaptive pipeline behaves more conservatively, staying closer to the main topic while still reducing some redundancy (relevance 53.6, diversity 60.6). The corresponding screenshot comparison is shown in Appendix A, Figure A.3.

Case 4: *Who is the US president in 2018*

This query is highly specific, factual, and time-sensitive, so the controller behaves as intended: it predicts high query clarity (85%), selects HNSW, sets $\lambda = 0.89$, and activates temporal handling with $s_T = 0.75$ and $\beta = 0.112$. Fixed MMR and the full adaptive pipeline are almost tied in this example ($F1 = 54.0$ for both), which is appropriate because the goal is to remain in a relevance-first regime rather than diversify aggressively. The remaining non-exact-answer items reflect a limitation of passage retrieval rather than a mistake in the control policy. The corresponding screenshot comparison is shown in Appendix A, Figure A.4.

7.8 Discussion

Taken together, the experiments support a trade-off view of diversified retrieval rather than a search for a single universally best ranking. Relevance-only retrieval preserves the highest raw query similarity but is clearly redundant. Fixed-MMR remains the strongest simple baseline on the current evaluation set.

Within the proposed adaptive family, L1 should be read only as a heuristic controller comparator rather than as a constituent stage of the deployed system. The deployed adaptive pipeline runs from L2 to L5, and under that reading L5 is the strongest learned-intent configuration. The main empirical contribution is therefore not universal dominance, but the demonstration that query-sensitive control can be combined with a serving-oriented ANN pipeline without collapsing ranking quality.

The query-type split and the qualitative cases make this behavioural claim more concrete. On ambiguous queries, the controller lowers λ , routes toward IVF, and often broadens coverage in a meaningful way, as in *spring*. At the same time, *mercury* shows the main failure mode of the approach: higher diversity is not automatically useful if it turns into topical drift. On specific or factual queries, the system behaves in the opposite direction. The *Fall of Berlin – 1945* and *Who is the US president in 2018* cases show that the controller can become more conservative, keeping the ranking closer to a relevance-first regime.

The index-level experiments also clarify the roles of the two ANN backends. On the current benchmark, HNSW behaves like the stronger focused retriever, while IVF exhibits the more expected recall–latency trade-off and remains useful as an exploratory candidate generator. This supports the backend-bias distinction used by the controller, but it does not validate the exact switch threshold at 0.4. That boundary remains an empirical heuristic in the present system, and a dedicated threshold-sensitivity analysis is future work.

The intent-model evaluation leads to a similarly qualified conclusion. On the independent 60-query test set, the predictor captures a meaningful ambiguity–specificity gradient, but it is much more reliable on clearly specific queries than on ambiguous or mid-range ones. This is consistent with both the synthetic supervision and the fact that retrieval specificity itself has a soft boundary. The intent model should therefore be understood as a moderately useful pre-retrieval control signal rather than as a fully reliable standalone model of user intent.

Several limitations follow directly from the current evaluation setup. The experiments use a 160-query test set and a single Wikipedia/Wikidata-style passage corpus of roughly 45k passages, so the reported behaviour is partly bounded by corpus construction, duplicate handling, and metadata quality. Freshness is implemented as a source-level prior rather than a document-level temporal model. In addition, several serving parameters — including the backend-switch boundary at 0.4, the relevance floor, and the cluster-penalty strength — remain hand-tuned heuristics. These limitations narrow the scope of the claims, but they do not change the central conclusion: query-aware diversification is behaviourally meaningful, and the final L5 configuration provides the strongest adaptive compromise among the proposed variants.

Chapter 8

Conclusions and Future Work

This dissertation studied top- k diversified vector search in a practical two-stage serving setting. Building on dense bi-encoder retrieval with Sentence-Transformers and MiniLM (Reimers and Gurevych, 2019; Wang et al., 2020), the system combines ANN candidate generation through IVF and HNSW (Jégou et al., 2011; Malkov and Yashunin, 2020) with diversified reranking based on Maximal Marginal Relevance (MMR) (Carbonell and Goldstein, 1998). The central contribution is per-query adaptivity: a learned intent regressor, factual fallback rules, and temporal detection are used to control both candidate generation and reranking.

The main technical contribution is not a new diversification objective in isolation, but a query-aware controller that acts before retrieval. It chooses the backend, adjusts search breadth and candidate-pool size, and then sets reranking strength according to the estimated ambiguity and temporal sensitivity of the query. The system also shows that diversified retrieval can be implemented efficiently in a serving-oriented setting. The incremental reranker reduces query-time memory from $O(N_c^2)$ to $O(N_c)$, while the relevance floor, quality-aware fallback, temporal bonus, and cluster-aware penalty provide practical safeguards against over-diversification.

Empirically, the project supports a measured conclusion rather than a universal dominance claim. Relevance-only retrieval is the most redundant, and Fixed-MMR remains the strongest simple baseline on the current evaluation set. Among the proposed adaptive variants, the full L5 pipeline provides the strongest overall adaptive compromise: it recovers much of the relevance lost by L2–L4 while remaining more diverse than the relevance-only baseline. The qualitative evidence is equally important: ambiguous queries are often broadened, whereas specific and fact-seeking queries are kept closer to a relevance-first regime.

At the same time, the limitations are clear. The *mercury* case shows that ambiguity-sensitive diversification can overshoot when the candidate pool is noisy. The intent model generalises only moderately, especially near the ambiguous/specific boundary. Freshness is represented by a lightweight source-level prior rather than a document-level temporal model. Several key control values — including the IVF/HNSW switch at 0.4 — remain empirically chosen heuristics rather than experimentally validated

optima or learned decisions. The system should therefore be understood as a practically deployable adaptive retriever, not as a final solution to diversified search.

These limitations suggest several directions for future work. First, the control policy for λ , β , backend switching, and search-width allocation could be learned from user interaction data rather than hand-mapped from intent and temporal signals. Second, the freshness component could be strengthened with document timestamps and time-decay functions. Third, a dedicated threshold-sensitivity study could test whether the current IVF/HNSW boundary at 0.4 is close to a good operating point or should be moved. Fourth, alternative diversification objectives such as xQuAD or determinantal point processes (Santos et al., 2010; Kulesza and Taskar, 2012) could be explored if their computational cost can be controlled. Finally, the retrieval controller could be extended to jointly optimise relevance, diversity, recall, and latency under an explicit serving budget.

Overall, the main conclusion of the project is that query-aware diversification is both behaviourally meaningful and practically feasible. Rather than applying a fixed amount of diversification to every request, the final system uses intent and temporal cues to decide when to explore broadly and when to remain conservative. That is the central lesson of the dissertation, and it is consistent with both the original motivation for MMR (Carbonell and Goldstein, 1998) and later work on intent-aware diversification (Santos et al., 2011).

Appendix A

Qualitative case study screenshots

This appendix collects the demo screenshots referenced in Section 7.7.

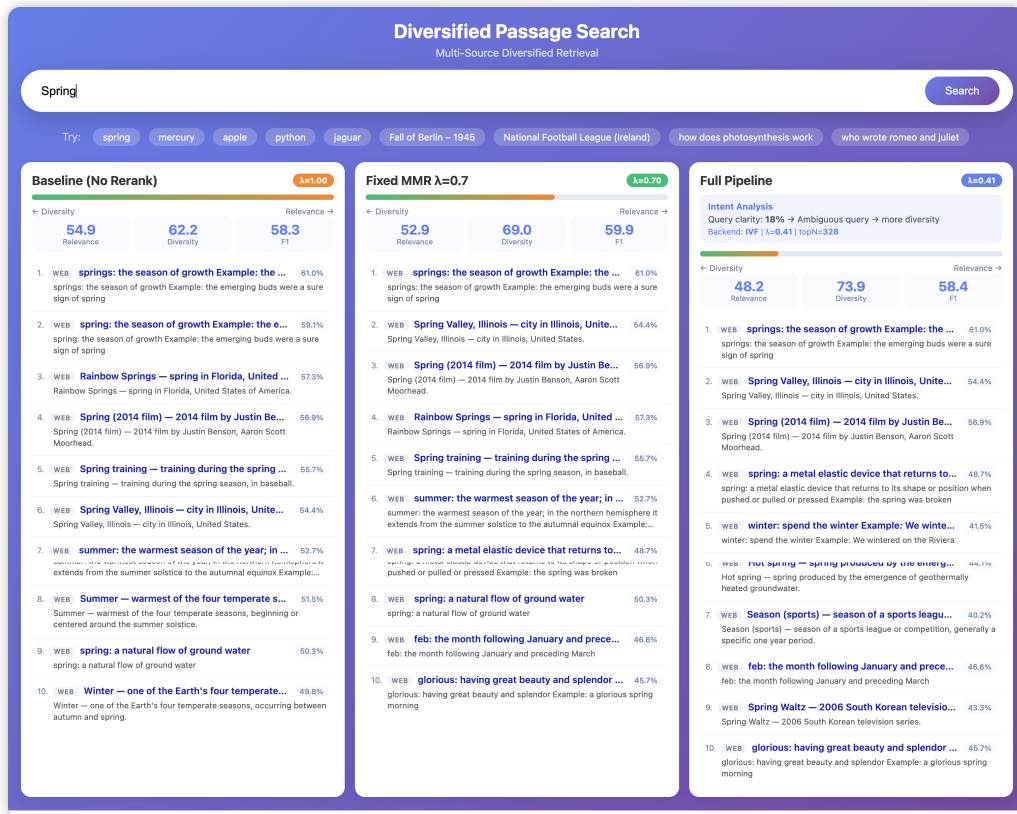
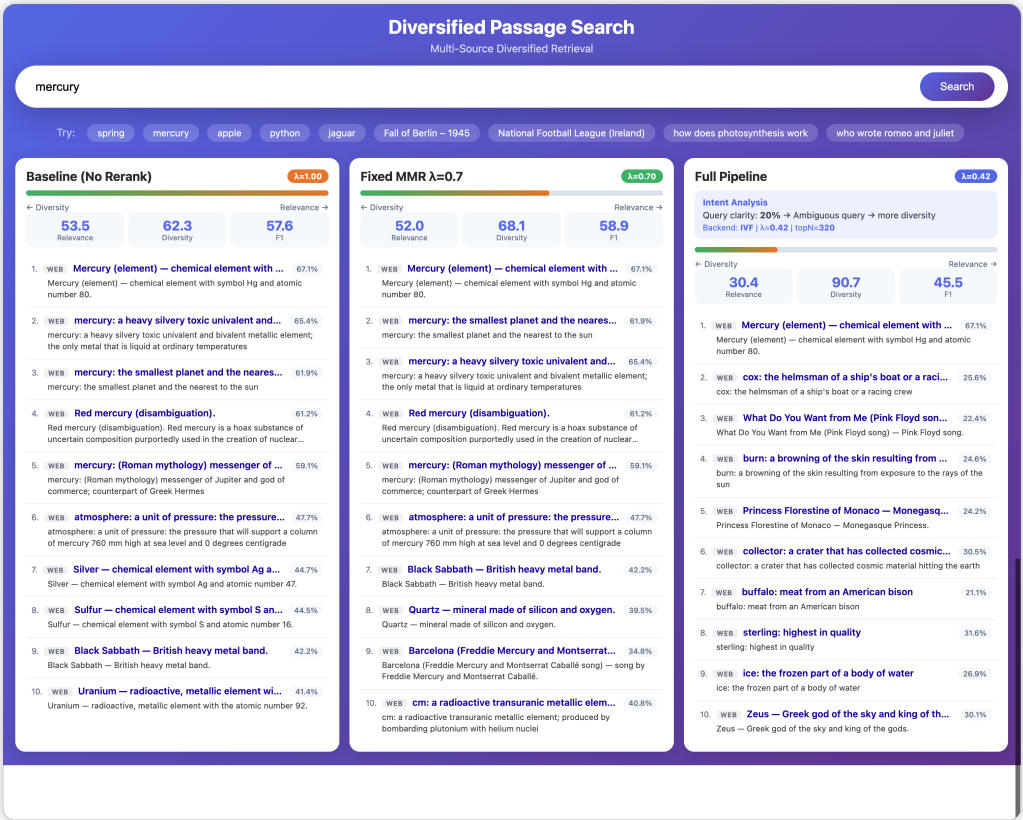


Figure A.1: Qualitative comparison for the ambiguous query *spring*.



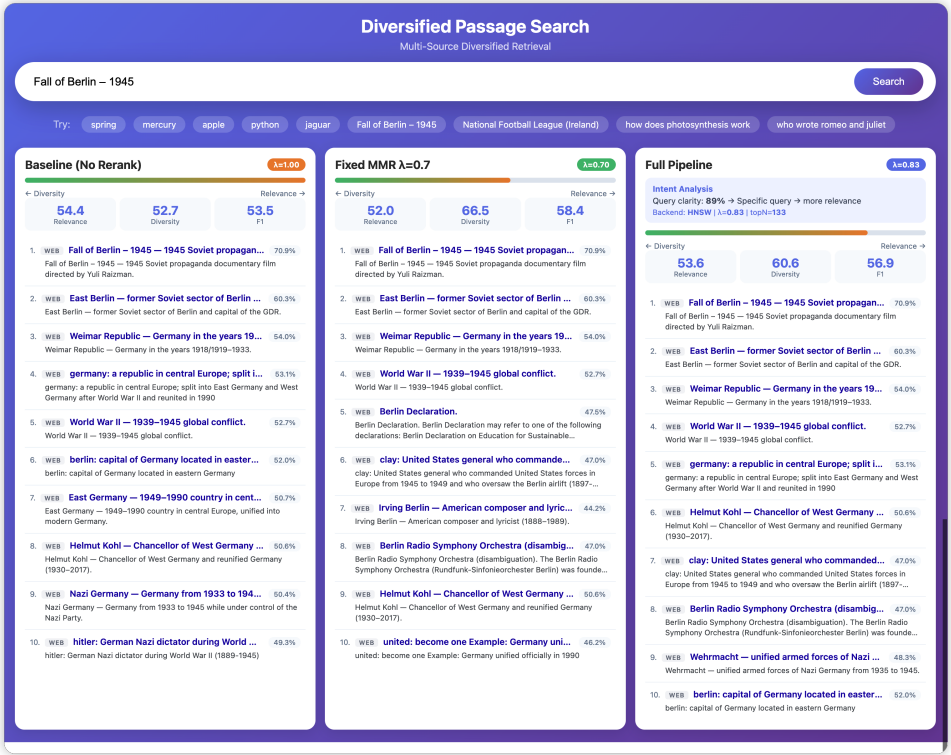
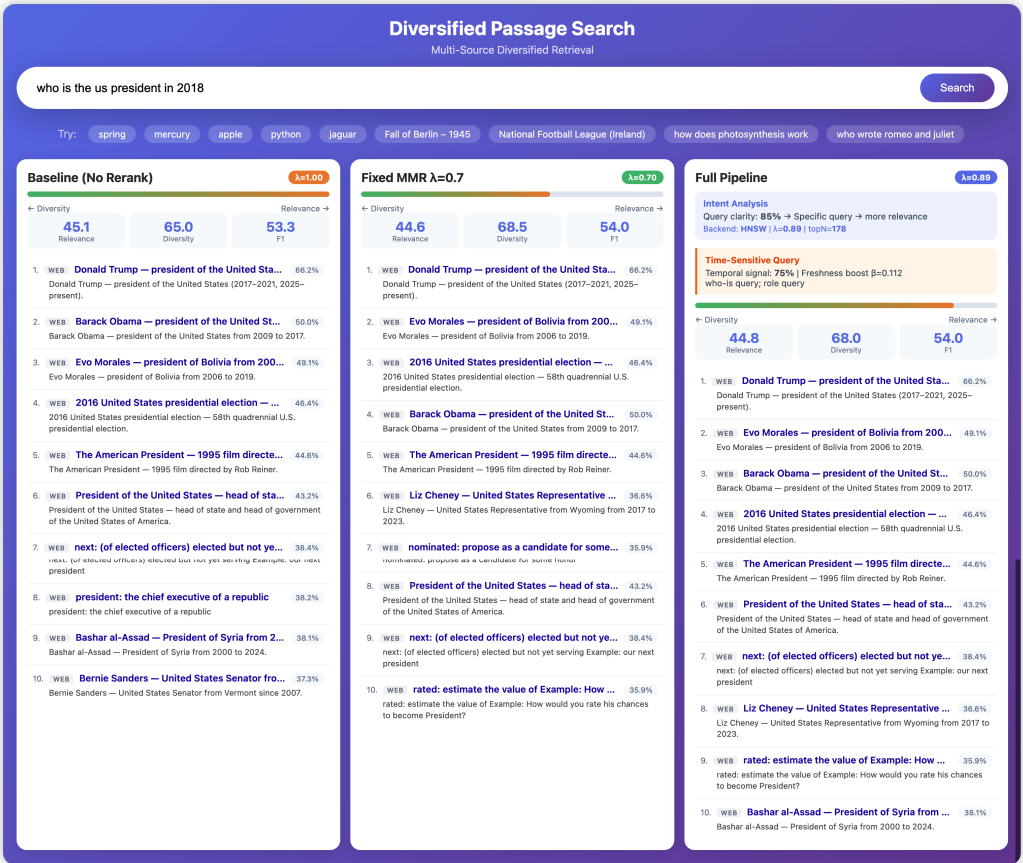


Figure A.3: Qualitative comparison for the specific query *Fall of Berlin – 1945*.



Bibliography

- Rakesh Agrawal, Sreenivas Gollapudi, Alan Halverson, and Samuel Jeong. Diversifying search results. In *Proceedings of the Second ACM International Conference on Web Search and Data Mining*, pages 5–14, 2009.
- David Arthur and Sergei Vassilvitskii. k -means++: The Advantages of Careful Seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.
- Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998.
- Olivier Chapelle, Shihao Ji, Ciya Liao, Emre Velipasaoglu, Larry Lai, and Su-Lin Wu. Intent-based diversification of web search results: Metrics and algorithms. *Information Retrieval*, 14(6):572–592, 2011.
- Charles L. A. Clarke, Maheedhar Kolla, Gordon V. Cormack, Olga Vechtomova, Azin Ashkan, Stefan Böttcher, and Ian MacKinnon. Novelty and diversity in information retrieval evaluation. In *Proceedings of the 31st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 659–666, 2008.
- Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, pages 191–198, 2016.
- Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *arXiv preprint arXiv:1702.08734*, 2017.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

- Alex Kulesza and Ben Taskar. Determinantal point processes for machine learning. *Foundations and Trends in Machine Learning*, 5(2–3):123–286, 2012.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.
- Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, 2019. Association for Computational Linguistics.
- Rodrygo L. T. Santos, Jie Peng, Craig Macdonald, and Iadh Ounis. Explicit search result diversification through sub-queries. In *Advances in Information Retrieval: 32nd European Conference on IR Research (ECIR 2010)*, pages 87–99, 2010.
- Rodrygo L. T. Santos, Craig Macdonald, and Iadh Ounis. Intent-aware search result diversification. In *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 595–604, 2011.
- Sentence-Transformers. all-MiniLM-L6-v2 model card. Hugging Face model card. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>, accessed 2026-03-28.
- Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, pages 5776–5788, 2020.
- Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.
- Cai-Nicolas Ziegler, Sean M. McNee, Joseph A. Konstan, and Georg Lausen. Improving recommendation lists through topic diversification. In *Proceedings of the 14th International Conference on World Wide Web*, pages 22–32, 2005.